

following command in the Erlang shell:

```
> edoc:files(["circleDoc.erl"], [{"dir", "doc"}]).
ok
```

The output files are written in the *doc* directory. The contents of the *doc* directory are as follows:

```
$ ls -l doc/
total 64
-rw-r--r-- 1 mtsouk staff 2887 May 25 18:17 circleDoc.html
-rw-r--r-- 1 mtsouk staff 57 May 25 18:17 edoc-info
-rw-r--r-- 1 mtsouk staff 2109 May 25 18:17 erlang.png
-rw-r--r-- 1 mtsouk staff 471 May 25 18:17 index.html
-rw-r--r-- 1 mtsouk staff 420 May 25 18:17 modules-frame.
html
-rw-r--r-- 1 mtsouk staff 1070 May 25 18:17 overview-
summary.html
-rw-r--r-- 1 mtsouk staff 327 May 25 18:17 packages-
frame.html
-rw-r--r-- 1 mtsouk staff 895 May 25 18:17 stylesheet.css
```

Figure 1 shows the central HTML page of the created documentation for the *circleDoc* module whereas Figure 2 shows the detailed documentation for both module functions.

Creating something useful in Erlang

Now, let's use OTP to program a server process. The line numbers are added in order to better refer to the Erlang code and need not be typed. The name of the module is *simpleServer*. I deleted all comments from the code in order to make it smaller but I always put comments inside the code I write.

The code for the Erlang program is as follows:

```
1 %% Programmer: Mihalis Tsoukalos
2 %% Date: Monday 27 May 2013
3
4 -module(simpleServer).
5 -behaviour(gen_server).
6 -define(SERVER, ?MODULE).
7 -record(state, {count}).
8
9 -export([
10   start_link/0,
11   stop/0,
12   say_hi/0,
13   get_count/0]).
14
15 -export([
16   init/1,
17   handle_call/3,
18   handle_cast/2,
19   handle_info/2,
20   terminate/2,
```

```
21   code_change/3]).
22
23 start_link() ->
24   gen_server:start_link(
25     {local, ?SERVER},
26     ?MODULE, [], []).
27
28 stop() ->
29   gen_server:cast(?SERVER, stop).
30
31 say_hi() ->
32   gen_server:cast(?SERVER, say_hi).
33
34 get_count() ->
35   gen_server:call(?SERVER, get_count).
36
37 init([]) ->
38   {ok, #state{count=0}}.
39
40 handle_call(get_count, _From, #state{count=Count}) ->
41   {reply, Count,
42    #state{count=Count+1}}.
43
44 handle_cast(stop, State) ->
45   {stop, normal, State};
46
47 handle_cast(say_hi, State) ->
48   io:format("Hi!\n"),
49   {noreply, #state{count=
50     State#state.count+1}}.
51
52 handle_info(Info, State) ->
53   error_logger:info_msg("-p-n", [Info]),
54   {noreply, State}.
55
56 terminate(_Reason, _State) ->
57   error_logger:info_msg("terminating-n", ok).
58 code_change(_OldVsn, State, _Extra) -> {ok, State}.
```

The key points of the *simpleServer* Erlang program are:

- Line 5: This tells Erlang that the module will implement the *gen_server* behaviour. A behaviour pre-defines the mechanisms you will use to interact with processes. It is a way of formalising common patterns.
- Line 6: This is a macro that instructs the Erlang compiler that every time it sees the word *?SERVER*, it should replace it with *?MODULE*. So, macros are simple text replacements.
- Line 7: This line is for keeping count of the number of total calls made.
- Line 31: This function prints a message on screen.
- Line 23: This function calls the *start_link* function of *get_server* in order to start up the process. It is useful for testing purposes, because it makes you see if the program actually runs.